

Three Mandatory Design Patterns Applied

1. Singleton Pattern:

Description:

The Singleton pattern is a creational design pattern that ensures a class has only one instance, while providing a global point of access to that instance.

Application in BookNest:

We applied the Singleton pattern in our **Database connection class**.

By using **Database::getInstance()->getConnection()**, we ensure that only a single database connection object is created and shared across the entire application.

2. MVC (Model-View-Controller) Pattern:

Description:

MVC is an architectural pattern that separates an application into three main logical components: the **Model**, the **View**, and the **Controller**.

This separation of concerns helps manage complexity and makes the code more scalable and maintainable.

Application in BookNest:

The entire platform is built on the MVC pattern:

- The **Models** handle database interactions and business logic (e.g., User.php, Book.php).
- The **Views** handle the HTML/UI presentation and display data to the user (e.g., books/show.php).
- The **Controllers** act as the middleman; they handle incoming HTTP requests, process data through the models, and return the appropriate views (e.g., BookController.php, AdminController.php).

3. Data Access Object (DAO) Pattern:

Application in BookNest:

We applied the **DAO** pattern heavily through our **Model classes**. Instead of writing **raw SQL** queries directly within the Controllers, these Model classes completely encapsulate all database interactions and **CRUD** operations.

For example, when a controller needs **to retrieve all users**, it simply calls **new User()->getAll()**. This keeps the controllers clean, secure, and completely unaware of the underlying SQL queries.